

Компьютерная графика

Практика 1: Рисуем треугольник

2026

О практиках

- Задания идут *последовательно*, полагаясь на предыдущие задания этой же практики
- Иногда задания *перекрывают* предыдущие задания, тогда нужно *закомментировать код предыдущего задания*, чтобы его тоже можно было проверить
- Некоторые задания занимают *больше одного слайда* :)

О практиках

- Вся инициализация спрятана от вас в мини-библиотеку `wgpu_app`
- Из неё вам будут нужны `app.device`, `app.queue`, `app.surface_format`
- Для C++ в репозитории есть маленькая библиотека математики – её использовать можно **только с третьей практики** :)

О практиках

- Мы постоянно будем загружать файлы с шейдерами, 3D моделями, текстурами, сценами
- Они будут лежать в корне конкретной практики
- Для удобства в C++ уже есть константа `projectRoot`, в Rust – `PROJECT_ROOT`
- В C++ есть вспомогательная функция `loadFile` в `<file_utils.hpp>`

Работа с WebGPU

- Функции создания WebGPU-объектов всегда принимают т.н. *descriptor* – небольшую структуру с описанием объекта
- В C++ вместо них можно передавать `nullptr`, если вам подойдёт дефолтная настройка
- На C++ их удобно инициализировать специальными макросами, например

```
WGPURenderPassColorAttachment colorAttachment =  
    WGPU_RENDER_PASS_COLOR_ATTACHMENT_INIT;
```

- На Rust можно использовать

```
wgpu::RenderPassColorAttachment {  
    // описание полей...  
    ..Default::default()  
}
```

Работа с WebGPU

- Так как `wgpu-native` – API для языка C, массивы в дескрипторах задаются как количество элементов + указатель на первый элемент, например

```
struct WGPUFragmentState {  
    // ...  
    size_t targetCount;  
    WGPUColorTargetState const * targets;  
};
```

Работа с WebGPU

- В C++ все созданные объекты нужно удалить, когда они не нужны, иначе будут утечки памяти

Работа с WebGPU

- В C++ все созданные объекты нужно удалить, когда они не нужны, иначе будут утечки памяти
- Это делается функциями вида `wgpuCommandBufferRelease`

Работа с WebGPU

- В C++ все созданные объекты нужно удалить, когда они не нужны, иначе будут утечки памяти
- Это делается функциями вида `wgpuCommandBufferRelease`
- На самом деле `wgpu` держит счётчик ссылок на объекты, так что делать *release* не страшно, даже если `wgpu` внутри всё ещё использует объект

Работа с WebGPU

- В C++ все созданные объекты нужно удалить, когда они не нужны, иначе будут утечки памяти
- Это делается функциями вида `wgpuCommandBufferRelease`
- На самом деле `wgpu` держит счётчик ссылок на объекты, так что делать *release* не страшно, даже если `wgpu` внутри всё ещё использует объект
- В Rust `wgpu` делает это автоматически (через Drop trait)

Задание 1

- Создаём **command encoder** – объект, позволяющий записать последовательность команд для выполнения на GPU
- Превращаем его в **command buffer** – собственно последовательность команд
- Отправляем их в **очередь команд** для исполнения – пока ни одной команды нет, так что ничего интересного не произойдёт
- Всё это в основном цикле рендеринга, сразу после `surface_texture.create_view`.
 - C++: `wgpuDeviceCreateCommandEncoder()`,
`wgpuCommandEncoderFinish()`, `wgpuQueueSubmit()`, не забываем про `release`
 - Rust: `device.create_command_encoder()`, `encoder.finish()`,
`queue.submit()`

Задание 2

- Через command encoder создаём **render pass**, который очистит наш экран фиксированным цветом
- Привязываем к нему **color attachment**, привязанный к нашему экрану (`target_view`)
- StoreOp – StoreOp, LoadOp – Clear, цвет выберите сами
- Всё это **ДО** `encoder.finish()`
 - C++: `WGPURenderPassColorAttachment,`
`WGPURenderPassDescriptor,`
`wgpuCommandEncoderBeginRenderPass(),`
`wgpuRenderPassEncoderEnd()`
 - Rust: `encoder.begin_render_pass()`

Задание 2



Задание 3

- Пишем наш первый шейдер: создаём файл `shader.wgsl` с магическим содержанием

```
@vertex
fn vertexMain(@builtin(vertex_index) vertexIndex: u32)
    -> @builtin(position) vec4f
{
    var positions = array<vec2f, 3>(
        vec2f( 0.0,  0.5),
        vec2f(-0.5, -0.5),
        vec2f( 0.5, -0.5),
    );
    return vec4f(positions[vertexIndex], 0.0, 1.0);
}

@fragment
fn fragmentMain() -> @location(0) vec4f {
    return vec4f(0.95, 0.35, 0.15, 1.0);
}
```

- (цвет во `fragmentMain` можно выбрать любой)

Задание 4

- При инициализации программы (до `while (running)` в C++, `WgpuApp::new` в Rust) загружаем наш файл с кодом шейдера и создаём **shader module**
- Он просто скомпилируется, но пока не используется – попробуйте сделать в нём ошибку (например, добавить лишнюю запятую) и увидеть, что компиляция падает с ошибкой
- C++: `WGPUShaderSourceWGSL, WGPUShaderModuleDescriptor, wgpuDeviceCreateShaderModule()`
- Rust: `device.create_shader_module(), wgpu::ShaderSource::Wgsl`

Задание 5

- Создаём **render pipeline** – основной объект, описывающий, что, как, и в каком формате рисовать
- Важно заполнить:

```
VertexState
    module = созданный нами shader module
    entry_point = "vertexMain"
FragmentState
    module = созданный нами shader module
    entry_point = "fragmentMain"
    targets = [{
        format = app.surface_format
        write_mask = ALL
    }]
PrimitiveState
    topology = TriangleList
```

- Остальное можно оставить дефолтами

Задание 5

- Пока мы не используем этот pipeline, так что ничего не произойдёт
- C++: `WGPUColorTargetState`, `WGPUFragmentState`,
`WGPURenderPipelineDescriptor`,
`wgpuDeviceCreateRenderPipeline()`
- Rust: `device.create_render_pipeline()`

Задание 6

- Используем созданный **render pipeline**: перед `encoder.end()` устанавливаем наш pipeline и вызываем рисование одного треугольника
- `vertex_count=3, instance_count=1, first_vertex` и `first_instance` по нулям
- Должен появиться цветной треугольник :)
- C++: `wgpuRenderPassEncoderSetPipeline,`
`wgpuRenderPassEncoderDraw()`
- Rust: `encoder.set_pipeline(), encoder.draw()`

Задание 6



Задание 7*

- Раскрашиваем треугольник в шахматном порядке
- Нужно как-то изменить `fragmentMain` в коде шейдера
- Полезные переменные и функции WGSL: `@builtin(position)`, `modf()`

Задание 7*

